UNIVERSIDADE FEDERAL DO RIO DE JANEIRO
ESCOLA POLITÉCNICA
DEPARTAMENTO DE ENGENHARIA ELETRÔNICA E DE COMPUTAÇÃO

PROPOSTA DE PROJETO DE GRADUAÇÃO

Aluno: Matheus Hoffmann Fernandes Santos
hoffmann@poli.ufrj.br

Orientador: A definir, Dr.
Coorientador: Pedro Veras Guimarães, Dr. (LabECO, Departamento de Engenharia Mecânica, UFSC)

1. TÍTULO

Otimização operacional do modelo de ondas WAVEWATCH III® para GPU.

2. ÊNFASE

Computação / Computação científica de alto desempenho.

3. TEMA

O WAVEWATCH III® (WW3) é um modelo numérico de previsão de ondas oceânicas desenvolvido pelos Centros Nacionais de Previsão Ambiental (NCEP) da Administração Nacional Oceânica e Atmosférica dos Estados Unidos (NOAA) e distribuído em código aberto (NOAA-EMC, 2026a; The WAVEWATCH III Development Group, 2019). O modelo representa o oceano por uma grade de pontos e descreve o estado do mar em cada ponto pelo espectro de ondas, que é a distribuição da energia das ondas entre frequências e direções. A cada passo de tempo, essa energia é propagada pela grade e modificada pelos chamados termos de fonte: a energia que o vento transfere às ondas, a troca de energia entre as próprias ondas e a perda por quebra. O WW3 pertence à classe dos modelos de terceira geração, que resolvem a equação de balanço da ação das ondas (energia dividida pela frequência) sem impor de antemão a forma do espectro (The WAMDI Group, 1988; Tolman, 1991). O código é escrito em Fortran e paralelizado com MPI e OpenMP, e um arquivo de *switches* define, na compilação, quais partes do código entram no executável, como a física, o esquema de propagação e as opções de entrada e saída (The WAVEWATCH III Development Group, 2019). No Brasil, o Laboratório de Engenharia e Ciências

Oceânicas da Universidade Federal de Santa Catarina (LabECO/UFSC) utiliza o WW3 em seu sistema de previsão operacional, como parte da Rede Nacional de Observação e Monitoramento Oceânico (ReNOMO) (LabECO/UFSC, 2026; UFSC, 2026).

O tema deste projeto é a **simulação paralela e a aceleração em GPU de um modelo espectral de ondas**, tendo como estudo de caso o WW3 na configuração operacional do LabECO. A questão que orienta o trabalho é quanto do tempo de execução (*wall-clock*) de um ciclo de previsão pode ser reduzido sem que os campos previstos difiram dos atuais mais do que as tolerâncias acordadas com o laboratório, e se uma GPU contribui para essa redução quando o estado do modelo permanece na memória da CPU. O trabalho parte de duas hipóteses, ambas verificáveis por medição. A primeira é que o tempo de execução atual ainda pode ser reduzido apenas com ajustes de compilação e de configuração, por uma margem maior que a variação entre execuções repetidas. A segunda é que, mesmo com o estado do modelo mantido na CPU, executar em GPU as rotinas mais custosas reduz o tempo total do ciclo. Isso só acontecerá se o tempo de cálculo economizado superar o custo de transferir os dados entre CPU e GPU a cada passo de tempo, custo que depende do volume copiado e da ligação entre CPU e GPU no nó do laboratório.

4. DELIMITAÇÃO

O trabalho se limita ao programa de grade única do WW3 (`ww3_shel`). Os pré- e pós-processadores do modelo, como o `ww3_grid` e o `ww3_ounf`, serão usados como estão, e apenas os seus arquivos de entrada poderão variar. Os casos de teste, em grades regulares e não estruturadas, virão do sistema de previsão operacional do LabECO, cuja configuração o laboratório entregará no início do cronograma. Grades não estruturadas são malhas de triângulos que permitem refinar a resolução junto à costa; em execuções paralelas, o WW3 as divide entre os processos com a biblioteca PDLIB (*Parallel Decomposition Library*). Cada caso reúne a grade, o arquivo de *switches*, as forçantes (os campos externos que alimentam o modelo, principalmente o vento) e a lista de saídas. No início do projeto, a configuração de cada caso, tal como roda hoje no laboratório, e a versão exata do código-fonte serão registradas e mantidas fixas (congeladas) até o fim do trabalho, para que todas as medições se

refiram sempre aos mesmos casos. Os testes de regressão oficiais do WW3 servirão apenas como instrumento de validação.

Quatro etapas fazem parte do escopo e serão executadas nesta ordem, cada uma validada por comparação com a execução de referência:

1. **Opções de compilação.** Serão escolhidos o compilador, as *flags* de otimização e a distribuição de processos MPI e *threads* OpenMP. No arquivo de *switches*, serão revistos a interpolação das forçantes e o modo de paralelização: memória compartilhada, MPI ou híbrido e, na grade não estruturada, a decomposição de domínio pela PDLIB. Esta etapa não altera o código-fonte.
2. **Configuração da execução.** Serão ajustados os passos de tempo, dentro do limite de estabilidade numérica dado pela condição de Courant–Friedrichs–Lewy (CFL), que, nos esquemas explícitos, relaciona o passo de tempo, o espaçamento da grade e a velocidade de propagação das ondas. Também serão revistas a frequência e a lista de campos de saída e a estratégia de reinício (*restart*). Esta etapa altera apenas as *namelists*, que são os arquivos de parâmetros lidos pelos programas do WW3.
3. **Refatoração dirigida em Fortran moderno, com apoio de um modelo de linguagem de grande porte (LLM, do inglês *large language model*).** Serão reescritas as rotinas que o perfil de desempenho indicar como dominantes, em geral a integração dos termos de fonte (`W3SRCE`) e a propagação na grade regular, com construções do Fortran 2008/2018 (`do concurrent`, interfaces explícitas, vetores contíguos) e sem alterar o algoritmo numérico. Um assistente de código baseado em LLM proporá a reescrita, rotina por rotina, como no trabalho de Koldunov et al. (2026) com o FESOM2, um modelo de circulação oceânica.
4. **Reescrita de *kernels* em C++/Kokkos e execução em GPU (NVIDIA H100).** O Kokkos é uma biblioteca C++ de portabilidade de desempenho que permite compilar o mesmo código-fonte para CPU e para GPUs de diferentes fabricantes (Trott et al., 2022). As rotinas que continuarem dominantes após a etapa 3 serão reescritas, uma a uma, como *kernels* Kokkos chamados a partir do Fortran. Se o termo de interação não linear, que representa a troca de energia entre ondas de frequências diferentes, ainda estiver

entre elas, será o primeiro, por ser autocontido e fácil de testar com um espectro sintético. O projeto não prevê migrar o modelo inteiro para a GPU: o estado do modelo continuará no Fortran, na memória da CPU, e o custo de transferir dados entre CPU e GPU a cada passo de tempo será medido e relatado.

A ordem das etapas faz parte da delimitação. As etapas 1 e 2 serão esgotadas antes de qualquer alteração no código, e a etapa 3 antes da etapa 4, de modo que a reescrita em C++/Kokkos só será feita em rotinas cujo custo, depois das etapas anteriores, tenha sido medido. Os termos de fonte são calculados ponto a ponto e, por isso, são os mesmos nos dois tipos de grade; nas etapas 3 e 4, serão tratados nas duas. Já a propagação na grade não estruturada usa rotinas próprias, ligadas à PDLIB, e será mantida como está.

Paralelamente, a NOAA desenvolve o WAVEWATCH IV (WW4), uma reescrita do modelo com núcleo em C++ (NOAA-EMC, 2026b; Tolman, 2025, 2026). Este projeto é independente desse desenvolvimento e não pretende contribuir com código para ele. Do WW4, adota apenas a estrutura de testes (níveis unitário e de integração, em GoogleTest) para os *kernels* reescritos, de modo que possam ser aproveitados em uma migração futura.

Ficam fora do escopo o programa multigrade (`ww3_multi`), o acoplamento com outros modelos, a assimilação de dados e qualquer alteração nas parametrizações físicas, isto é, nas formulações dos termos de fonte.

5. JUSTIFICATIVA

Os modelos de ondas são forçados pelo vento previsto por modelos atmosféricos. Como esses modelos são governados por equações não lineares, a previsão do vento, e com ela a das ondas, perde qualidade à medida que o horizonte de previsão aumenta. Campos et al. (2020) compararam o sistema de previsão por conjunto do NCEP com medidas de quatro satélites altímetros, que registram a altura das ondas a partir do espaço, e mostraram que o erro cresce ao longo dos dias de previsão. Por esse motivo, os sistemas operacionais atualizam a previsão várias vezes por dia e recorrem à previsão por conjunto (*ensemble*), na qual o modelo é rodado diversas vezes com pequenas perturbações nas condições iniciais e nas forçantes. A média das rodadas tende a errar menos que uma rodada isolada (Farina, 2002). No sistema

do NCEP, a média do conjunto reduziu a componente aleatória do erro da altura significativa de onda, que corresponde aproximadamente à média do terço mais alto das ondas, de 1,8 m, na rodada de controle (sem perturbações), para 1,3 m (Campos et al., 2020).

A frequência dos ciclos, a resolução e o número de grades aninhadas e de membros do conjunto são limitados pelo tempo de execução de cada previsão e pelos recursos computacionais disponíveis. O NCEP, por exemplo, roda o GFS-Wave, seu sistema global de previsão de ondas, quatro vezes ao dia (00, 06, 12 e 18 UTC), em grades globais de 0,16° e 0,25° com grades regionais mais finas aninhadas, além do GEFS-Wave, a versão por conjunto, com 30 membros a 0,25°, ambos com horizonte de 384 h (NOAA/NWS/NCEP/EMC, 2026).

O mesmo vale para o LabECO. A ReNOMO, financiada pela chamada CNPq/MCTI/Finep 062/2022, integra observação, controle de qualidade e produtos operacionais para o Atlântico Sul e a costa brasileira (LabECO/UFSC, 2026). Para o campo de ondas que alimenta esses produtos, reduzir o tempo de cada ciclo permite entregar a previsão mais cedo, usar uma grade mais fina no mesmo prazo ou rodar membros de conjunto para quantificar a incerteza.

O WW3 foi concebido para os supercomputadores de duas décadas atrás, e a própria NOAA reconhece que os conceitos de paralelização e as estruturas de dados do modelo, herdadas da transição para o Fortran 90, não acompanharam a evolução do *hardware* (Tolman, 2025). A computação de alto desempenho vem migrando de processadores de uso geral para nós heterogêneos, com muitos núcleos e aceleradores como as GPUs. Essa mudança tem levado os códigos de previsão do tempo, do oceano e do clima a rever sua estrutura (Bauer et al., 2021) e motivou o desenvolvimento de camadas de portabilidade de desempenho como o Kokkos (Trott et al., 2022).

As etapas 1 e 2 vêm primeiro porque não exigem mudanças no código. Além disso, segundo o manual do modelo, parte do desempenho depende de escolhas feitas fora do código-fonte. O arquivo de *switches* e as opções do compilador determinam qual código é compilado e de que forma, e a distribuição de processos MPI e *threads* OpenMP determina quanto tempo se gasta trocando espectros entre processos em vez de calculá-los (The WAVEWATCH III Development Group, 2019). Em uma

instalação configurada uma única vez, na implantação do modelo, e não revista desde então, parte do tempo de execução pode se dever à configuração, e não ao código. Essas etapas custam pouco, apenas horas de máquina, e produzem os dois instrumentos de que as etapas seguintes dependem: um *benchmark* reprodutível, que mede o tempo de execução sempre da mesma forma, e um perfil de desempenho, que mostra em quais rotinas esse tempo é gasto. Sem esses dados, não há como saber quanto tempo ainda se pode economizar, nem em quais rotinas. Por isso, o código só será reescrito depois de obtidos o *benchmark* e o perfil.

A etapa 3 se justifica pela estrutura do código. O perfil publicado por Ikuyajolu et al. (2023) mostra que a integração dos termos de fonte domina o tempo de execução do WW3, seguida da propagação, e que as rotinas envolvidas usam grandes vetores locais e variáveis globais declaradas em módulos, o que dificulta tanto a vetorização quanto a execução em GPU. Reescrever essas rotinas em Fortran moderno, preservando o algoritmo e a aritmética, é um trabalho de engenharia de risco conhecido, com ganho diretamente mensurável no modelo que o laboratório opera. Além disso, prepara a etapa seguinte, já que uma rotina com interface explícita e dados contíguos em memória pode ser substituída por um *kernel*.

A etapa 4 responde a uma pergunta do próprio laboratório: se uma GPU H100 pode acelerar o modelo em operação. A distribuição oficial do WW3 não oferece execução em GPU (NOAA-EMC, 2026a). O único *port* publicado de que se tem notícia levou os termos de fonte para a GPU com diretivas OpenACC e obteve um ganho de cerca de 1,3 vez por nó, limitado pela transferência de dados entre CPU e GPU a cada passo de tempo (Ikuyajolu et al., 2023). Já os autores do WAM6-GPU, versão para GPU de outro modelo de ondas de terceira geração, reestruturaram o modelo inteiro para manter todos os campos na memória da GPU e obtiveram um ganho de uma ordem de grandeza em um nó com várias GPUs (Yuan et al., 2024). Os dois resultados indicam que o ganho depende de os dados permanecerem na GPU: quando o estado do modelo fica na CPU e precisa ser copiado a cada passo, a transferência limita o ganho. Ainda assim, este projeto manterá o estado no Fortran, por três motivos. Migrar o modelo inteiro não cabe no prazo de um projeto de graduação; a substituição rotina por rotina permite validar cada *kernel* isoladamente, em comparação com o código original; e medir o custo

dessa transferência no H100 já é um resultado útil para o laboratório. O que pode tornar o resultado deste projeto diferente do de Ikuyajolu et al. (2023) é o volume copiado por chamada, que a etapa 3 reduzirá ao tornar explícitas as entradas e as saídas de cada rotina, no lugar das variáveis globais de módulo. Mesmo assim, o resultado da etapa 4 pode ser negativo, isto é, um *kernel* pode ficar mais rápido na GPU sem que o ciclo fique mais curto. A tradução seguirá o método de Koldunov et al. (2026), que traduziram cerca de 74 mil linhas de Fortran do FESOM2 para C e depois para C++/Kokkos em semanas, com um assistente de código baseado em LLM dirigido por especialistas, tradução literal em duas etapas e validação em níveis sucessivos. Esse trabalho ainda não foi revisado por pares. O critério para levar um *kernel* à operação será o mesmo das demais etapas: ganho medido e resultados dentro das tolerâncias.

Estado dos testes do WW3. A validação proposta se apoia na infraestrutura de testes do próprio modelo, tal como estava no ramo `develop` (versão 7.14) em setembro de 2026 (NOAA-EMC, 2026a). O WW3 tem 62 casos de teste de regressão em `regtests/`, executados por uma matriz de *scripts* que compila combinações de *switches*, compara arquivo a arquivo, com `cmp` e `diff`, as saídas de duas execuções e classifica cada caso como idêntico ou não. A matriz inclui variantes que verificam a reprodutibilidade bit a bit após um reinício e com números diferentes de *threads* e de processos. Não há comparação com tolerância: qualquer diferença numérica é registrada como falha, e cabe a quem revisa julgar se ela é aceitável. Os testes unitários, introduzidos em 2023, cobrem apenas entrada e saída (cinco programas, via CTest). Esse quadro define como o projeto validará cada etapa. Nas etapas 1 e 2, a matriz do próprio modelo validará as opções que não alteram a aritmética, das quais se exige reprodução bit a bit. As opções que alteram a solução exigem um comparador campo a campo com tolerâncias, e as etapas 3 e 4 exigem, além dele, testes unitários por rotina. O WW3 não tem nenhum dos dois, e o projeto os construirá.

Estado do WW4. O repositório do WAVEWATCH IV foi criado em novembro de 2025 e, em 7 de outubro de 2026, ainda não tinha nenhuma versão publicada (NOAA-EMC, 2026b). A Fase II do projeto, que tratou de linguagens, governança e arquitetura, foi concluída em março de 2026 (Tolman, 2026). Em 7 de

outubro de 2026, o núcleo em C++ ainda não tinha termos de fonte nem propagação; uma alteração em revisão acrescentaria apenas a seleção, durante a execução e sem recompilar, do esquema de propagação e dos termos de fonte, todos ainda sem implementação (NOAA-EMC, 2026b). Nessa data, a estratégia de testes previa quatro níveis em GoogleTest, dos quais só existiam os dois primeiros, unitário (L1) e de integração (L2); os níveis funcional (L3) e de regressão (L4), que substituiriam a matriz do WW3, ainda não tinham sido escritos (NOAA-EMC, 2026b). A discussão sobre a arquitetura para CPU e GPU, que propunha o Kokkos como camada de abstração de dados, foi encerrada sem decisão registrada, e nenhum arquivo do código mencionava o Kokkos (NOAA-EMC, 2026b). Segundo o relatório da Fase I, o primeiro lançamento público estava previsto para meados de 2027, e a maior parte do suporte ao WW3 só será encerrada quando o WW4 estiver maduro (Tolman, 2025). Daí decorrem duas consequências: o WW3 continuará sendo o modelo operacional do laboratório por alguns anos, o que justifica otimizá-lo, e os *kernels* reescritos aqui, em Kokkos e com testes L1 e L2 no padrão do WW4, podem servir de evidência para a escolha da arquitetura de CPU e GPU do WW4 sem duplicar o desenvolvimento do modelo.

6. OBJETIVO

O objetivo geral é reduzir o tempo de execução do ciclo operacional do WAVEWATCH III mantido pelo LabECO/UFSC para a ReNOMO, com ganhos medidos e reprodutíveis, sem que os resultados difiram dos atuais mais do que as tolerâncias acordadas com o laboratório. Um único critério (ganho medido e resultados dentro das tolerâncias) decidirá também se *kernels* em C++/Kokkos executados em uma GPU NVIDIA H100 passam a fazer parte da configuração operacional.

Os objetivos específicos são:

1. Registrar e congelar a configuração operacional de cada caso (versão do código, *switches*, *namelists*, grade, forçantes, saídas e *hardware*) e construir um *benchmark* reprodutível da execução de referência, com uma métrica definida: o tempo de execução por hora de previsão.
2. Levantar o perfil de desempenho da execução de referência por rotina e por fase (termos de fonte, propagação, comunicação, entrada e saída), com um e com vários processos MPI.

3. Quantificar o ganho das etapas de compilação, de configuração e de refatoração em Fortran, cada uma acompanhada da verificação de concordância com a referência, e entregar ao laboratório a melhor configuração na forma de um *build* documentado.
4. Construir a infraestrutura de validação que o WW3 não tem: um comparador campo a campo com tolerâncias versionadas e testes unitários por rotina com entradas capturadas do caso operacional, integrados à matriz de regressão do modelo.
5. Reescrever em C++/Kokkos, na ordem indicada pelo perfil, as rotinas que continuarem dominantes, validá-las em CPU em relação ao Fortran original, medir o ganho em GPU (H100) e decidir, com base no ganho e na concordância dos resultados, se elas entram na configuração operacional.
6. Publicar ferramentas, resultados e recomendações no repositório do projeto, de forma que o laboratório possa repetir as medições, respeitada a política de divulgação descrita na metodologia.

7. METODOLOGIA

O projeto seguirá o princípio de medir antes de otimizar e avançará em ciclos de teste e validação. Em todos os ciclos serão medidas duas grandezas: o tempo de execução, total e por fase do laço temporal, expresso como tempo de execução do `ww3_shel` por hora de previsão, com o pós-processamento registrado à parte; e a concordância dos resultados, avaliada pela diferença entre os campos produzidos pela execução modificada e pela execução de referência. Cada medida de tempo será repetida pelo menos cinco vezes, em nó dedicado, e o relatório trará a mediana e o intervalo interquartil. Em cada ciclo, um único elemento será alterado (uma opção, um parâmetro ou uma rotina), e os testes serão executados do mais barato para o mais caro: primeiro a rotina isolada, quando houver alteração de código, depois uma grade simplificada e, por fim, o caso operacional. Os resultados serão então comparados com a referência; a partir da etapa 4, o projeto comparará também, na grade simplificada, a execução em CPU com a execução em GPU do mesmo código. A alteração aprovada será incorporada e passará a ser a base do ciclo seguinte, e a reprovada voltará para a reescrita ou será descartada.

Casos de referência e grades simplificadas. Os casos operacionais do

laboratório, em grade regular e em grade não estruturada, serão capturados como testes autocontidos, com arquivos de entrada, forçantes para um período fixo, arquivo de *switches*, *namelists* e resultados de referência. Para cada tipo de grade será definida também uma grade simplificada, um domínio reduzido derivado do caso operacional ou do teste de regressão oficial mais próximo, que roda em poucos minutos tanto em CPU quanto em GPU e será usada em todos os ciclos de teste. A grade simplificada serve apenas para acelerar os ciclos e não substitui o caso operacional em nenhuma medição relatada. O *script* de *benchmark* registrará o tempo por fase, o número de processos e de *threads*, o compilador e suas opções e a versão do código, e produzirá uma tabela que poderá ser anexada aos relatórios.

Perfil de desempenho. O perfil de desempenho da execução de referência será levantado com duas ferramentas complementares. O `gprof` instrumenta o executável na compilação (opção `-pg`) e gera o grafo de chamadas, com o tempo acumulado em cada rotina e nas rotinas que ela chama (Graham; Kessler; McKusick, 1982). O `perf`, distribuído com o núcleo do Linux, amostra periodicamente a execução por meio dos contadores de desempenho do processador, sem recompilar o modelo, e atribui o tempo a rotinas e a linhas do código (Linux Kernel Organization, 2026). O primeiro fornece a estrutura de chamadas, e o segundo confirma os tempos sem a interferência da instrumentação. As medições serão feitas com um, quatro e dezesseis processos MPI, registrando-se à parte o tempo de comunicação. O resultado será uma tabela de tempo por rotina, agrupada pelas fases do laço temporal, que definirá a ordem da refatoração e servirá de base para todas as medições seguintes.

Etapas 1 e 2. Na etapa 1, as opções de compilação serão exploradas como uma matriz de compilador, opções e distribuição paralela, que inclui, na grade não estruturada, a compilação com a PDLIB. Na etapa 2, o ponto de partida será um único conjunto de *namelists* por caso, o operacional, congelado no início do projeto e usado também em todas as demais etapas. A partir dele, o projeto variará, um de cada vez, apenas os parâmetros listados na delimitação, que afetam o desempenho sem alterar as parametrizações físicas. O resultado será, no máximo, um segundo conjunto de *namelists*, documentado e proposto ao laboratório.

Etapas 3 e 4. Na etapa 3, a refatoração em Fortran tratará as rotinas do

topo do perfil, uma de cada vez, em um *fork* do código que mantém a rotina original no mesmo executável, selecionável em tempo de execução. Só então começará a etapa 4, restrita às rotinas que o perfil atualizado ainda mostrar como dominantes.

Critérios de concordância e ordem dos testes. Nas etapas 1 e 2, a matriz de regressão do próprio WW3 será usada como está. As opções que não alteram a aritmética do modelo, como a distribuição de processos e de *threads* e as *flags* que, com o mesmo compilador, preservam a semântica de ponto flutuante, deverão reproduzir a referência bit a bit, arquivo a arquivo, nos casos operacionais e nas grades simplificadas. As que alteram a solução ou os arquivos serão avaliadas pelo comparador campo a campo descrito a seguir, sobre os campos e os instantes comuns às duas execuções. Entram nesse grupo a troca de compilador, as *flags* que reordenam operações ou fundem multiplicação e soma em uma só instrução (FMA, do inglês *fused multiply-add*), a interpolação das forçantes, o passo de tempo, a frequência das saídas e a lista de saídas. As tolerâncias desse comparador serão definidas pelo grupo de pesquisa do LabECO para cada campo, como a altura significativa de onda, o período de pico e a direção de pico, que são o período e a direção das ondas que concentram mais energia. Nas etapas 3 e 4, em que o código muda, os testes seguirão dois níveis, nesta ordem. Primeiro, antes de qualquer execução do modelo completo, testes unitários por rotina, que a matriz não oferece, executarão a rotina original e a reescrita com entradas capturadas do caso operacional e compararão os vetores de saída em memória, o que detecta um erro em segundos e já aponta a rotina responsável. Depois, o comparador campo a campo lerá as saídas em NetCDF de duas execuções completas, primeiro na grade simplificada e depois no caso operacional, e aplicará as tolerâncias registradas em um arquivo versionado. Como o arquivo NetCDF é o produto final da cadeia, esse teste vem por último. Uma alteração reprovada em qualquer nível não será incorporada: voltará para a reescrita e repetirá o ciclo desde os testes unitários.

Interoperabilidade e testes dos *kernels*. Cada *kernel* em C++/Kokkos será chamado a partir do Fortran por uma interface `ISO_C_BINDING`/`bind(C)`, e a rotina original continuará no executável, selecionável pela mesma opção, de modo que a matriz de regressão e o comparador possam testar os dois caminhos sem recompilar. A organização dos `Views` do Kokkos reproduzirá a dos vetores espectrais

do WW3, para que a chamada em CPU não copie dados. A tradução seguirá o método aplicado ao FESOM2 (Koldunov et al., 2026). Primeiro, será feita uma tradução literal para C, verificada em relação ao Fortran com entradas capturadas. Em seguida, será escrita uma versão em Kokkos, cujo *backend* serial deverá reproduzir o C bit a bit. Só então o *backend* CUDA será executado no H100 e comparado com a execução em CPU pelo erro máximo e pela raiz do erro quadrático médio (REQM) de cada campo, primeiro na grade simplificada e depois no caso operacional, com registro do volume de dados transferido entre CPU e GPU a cada passo de tempo. Os testes seguirão a estrutura do WW4. No nível L1, cada *kernel* terá um teste unitário em GoogleTest a partir de um espectro sintético de mar gerado pelo vento, cuja forma empírica foi obtida no projeto JONSWAP (*Joint North Sea Wave Project*) e descrita por Hasselmann et al. (1973), com tolerâncias declaradas e justificadas. No nível L2, a grade simplificada e o caso operacional serão executados novamente, e o comparador campo a campo confrontará o WW3 original com o WW3 com Kokkos. A decisão de levar um *kernel* à operação será tomada com o LabECO com base em duas evidências: a tabela de tempos do caso operacional, com e sem o *kernel*, e o relatório de concordância.

Atividades com apoio de LLM. Um assistente de código baseado em LLM, dirigido pelo aluno, será usado em quatro atividades: a leitura e a documentação das rotinas dominantes do WW3, para levantar suas dependências de variáveis globais antes da reescrita; a proposta de refatoração em Fortran moderno, rotina por rotina, na etapa 3; a tradução literal de Fortran para C e de C para C++/Kokkos na etapa 4, segundo o método de Koldunov et al. (2026); e a geração dos testes unitários e dos *scripts* de captura de entradas e de comparação. Nenhum código gerado pelo assistente será aceito sem cumprir os mesmos critérios de concordância exigidos do código escrito à mão, e o aluno e o coorientador responderão pela revisão do código e pelas evidências de concordância. Os *prompts*, o *diff* revisado e as evidências de concordância serão versionados junto com o código. O relatório final registrará, para cada atividade, o que o assistente produziu, o que foi corrigido à mão e o que foi descartado, como avaliação do próprio método.

Ferramentas e reprodutibilidade. O código, as ferramentas e os testes serão versionados no repositório público `ww3-gpu` (Hoffmann, 2026), que fornece a

cadeia de ferramentas com versões fixadas, os *scripts* de compilação e de teste de regressão, o modelo como submódulo em revisão fixa e a integração contínua, de modo que qualquer medição possa ser repetida pelo laboratório em suas próprias máquinas. A configuração operacional e os dados do LabECO, assim como os resultados ainda não publicados, ficarão fora do repositório público, em local de acesso restrito definido pelo laboratório. Esses dados e resultados permanecerão restritos até a defesa do projeto de graduação ou até a submissão de um artigo pelo grupo de pesquisa, o que vier primeiro, a menos que os orientadores decidam de outra forma. Os testes que dependem de entradas capturadas do caso operacional ficarão junto com esses dados; a integração contínua pública usará apenas os testes de regressão oficiais e espectros sintéticos. Os resultados serão registrados em tabelas e gráficos gerados por *scripts*.

Entregas e validação. As entregas são o registro da configuração, as tabelas de *benchmark* e de perfil, o melhor *build* documentado, o comparador campo a campo e os testes unitários, os *kernels* Kokkos com as evidências de concordância e de tempo, e o relatório final com a recomendação de operação. A validação científica caberá ao grupo de pesquisa do LabECO, que define as tolerâncias e revisa as evidências de concordância; a validação de engenharia, aos testes automatizados do repositório e a reexecuções independentes no laboratório.

Riscos. O principal risco é uma divergência dos resultados que passe despercebida após uma reescrita, mitigada pelos critérios de concordância e pelos testes por rotina. O segundo é a etapa 4 consumir o tempo das anteriores, mitigado pela ordem fixa das etapas e pela regra de só reescrever rotinas cujo custo, após as etapas anteriores, tenha sido medido. O terceiro é o ganho em GPU ficar limitado pela transferência de dados enquanto o estado permanecer no Fortran. Nesse caso, o resultado será a medida desse limite, a recomendação de não operar em GPU nessa configuração e a indicação do trabalho seguinte, que é manter os dados na GPU entre um passo de tempo e outro. O quarto é o atraso na entrega dos casos do LabECO ou no acesso ao H100. Nesse caso, as etapas 1 e 2 começarão provisoriamente com o teste de regressão oficial mais próximo da configuração operacional, e esses números não entrarão no relatório como resultado das etapas. As medições da etapa 4 serão feitas na GPU disponível, com a ressalva de que não valem diretamente para

o H100.

8. CRONOGRAMA

O projeto ocupará dois períodos letivos, e as datas serão ajustadas com os orientadores.

Tabela 1: Cronograma do projeto de graduação.

	2026			2027				
Atividade	out	nov	dez	jan	fev	mar	abr	maio
Revisão bibliográfica; registro da configuração operacional (grades regular e não estruturada)	■							
Benchmark reprodutível, grades simplificadas, comparador campo a campo e perfil de desempenho		■						
Etapa 1: opções de compilação (compilador, *flags*, *switches*, paralelização, PDLIB)			■					
Etapa 2: configuração da execução; testes unitários por rotina				■				
Etapa 3: refatoração em Fortran moderno das rotinas dominantes					■			
Etapa 4: *kernels* em C++/Kokkos; concordância em CPU e medição no H100						■		
Decisão de operação com o LabECO; relatório final e revisão							■	
Defesa								♦

■ mês da atividade; ♦ marco, a partir do mês indicado

Referências Bibliográficas

BAUER, Peter *et al.* The digital revolution of Earth-system science. **Nature Computational Science**, v. 1, n. 2, p. 104–113, 2021.

CAMPOS, Ricardo Martins *et al.* Global assessments of the NCEP Ensemble Forecast System using altimeter data. **Ocean Dynamics**, v. 70, n. 3, p. 405–419, 2020.

FARINA, Leandro. On ensemble prediction of ocean waves. **Tellus A: Dynamic Meteorology and Oceanography**, v. 54, n. 2, p. 148–158, 2002.

GRAHAM, Susan L.; KESSLER, Peter B.; MCKUSICK, Marshall K. gprof: a call graph execution profiler. **ACM SIGPLAN Notices**, v. 17, n. 6, p. 120–126, 1982.

HASSELMANN, Klaus *et al.* Measurements of wind-wave growth and swell decay during the Joint North Sea Wave Project (JONSWAP). **Ergänzungsheft zur Deutschen Hydrographischen Zeitschrift, Reihe A**, n. 12, p. 1–95, 1973.

HOFFMANN, Matheus. **ww3-gpu: an open lab for WAVEWATCH III operational optimisation and GPU kernels**. 2026. Disponível em: <https://github.com/h0ffmann/ww3-gpu>. Acesso em: 18 set. 2026.

IKUYAJOLU, Olawale James *et al.* Porting the WAVEWATCH III (v6.07) wave action source terms to GPU. **Geoscientific Model Development**, v. 16, p. 1445–1458, 2023.

KOLDUNOV, Nikolay V. *et al.* **An Ocean Model Ported by a Large Language Model: Experience and Lessons from FESOM2 (Fortran to C to C++/Kokkos)**. arXiv:2606.11356, 2026. Disponível em: <https://arxiv.org/abs/2606.11356>. Acesso em: 1 out. 2026.

LABECO/UFSC. **ReNOMO – Rede Nacional de Observação e Monitoramento Oceânico**. 2026. Disponível em: <https://labeco.ufsc.br/renomo/>. Acesso em: 18 set. 2026.

LINUX KERNEL ORGANIZATION. **perf: Linux profiling with performance counters**. 2026. Disponível em: <https://perf.wiki.kernel.org/>. Acesso em: 18 set. 2026.

NOAA-EMC. **WAVEWATCH III source code repository: develop branch, v7.14**. 2026a. Disponível em: <https://github.com/NOAA-EMC/WW3>. Acesso em: 15 set. 2026.

NOAA-EMC. **WAVEWATCH IV (WW4) repository**. 2026b. Disponível em: <https://github.com/NOAA-EMC/WW4>. Acesso em: 7 out. 2026.

NOAA/NWS/NCEP/EMC. **Operational wave models**. 2026. Disponível em: <https://www.emc.ncep.noaa.gov/emc/pages/numerical_forecast_systems/wavemodels.php>. Acesso em: 16 set. 2026.

THE WAMDI GROUP. The WAM Model—A Third Generation Ocean Wave Prediction Model. **Journal of Physical Oceanography**, v. 18, n. 12, p. 1775–1810, 1988.

THE WAVEWATCH III DEVELOPMENT GROUP. **User manual and system documentation of WAVEWATCH III version 6.07**. College Park, MD: NOAA/NWS/NCEP/M 2019. (Tech. Note, 333).

TOLMAN, Hendrik L. A Third-Generation Model for Wind Waves on Slowly Varying, Unsteady, and Inhomogeneous Depths and Currents. **Journal of Physical Oceanography**, v. 21, n. 6, p. 782–797, 1991.

TOLMAN, Hendrik L. **The WAVEWATCH III Software Modernization Project: Phase I report**. College Park, MD: NOAA/NWS/NCEP, 2025. (Office Note, 525). Disponível em: <https://doi.org/10.25923/h7j3-1h25>. Acesso em: 15 set. 2026.

TOLMAN, Hendrik L. **The WAVEWATCH IV Project: Phase II report**. College Park, MD: NOAA/NWS/NCEP, 2026. (Office Note, 528). Disponível em: <https://doi.org/10.25923/0wyp-9f39>. Acesso em: 15 set. 2026.

TROTT, Christian R. *et al.* Kokkos 3: Programming model extensions for the exascale era. **IEEE Transactions on Parallel and Distributed Systems**, v. 33, n. 4, p. 805–817, 2022.

UFSC. **Laboratório de Engenharia e Ciências Oceânicas (LabECO)**. 2026. Disponível em: <https://emc.ufsc.br/portal/laboratorios-2/labeco/>. Acesso em: 18 set. 2026.

YUAN, Ye *et al.* Towards a real-time modeling of global ocean waves by the fully GPU-accelerated spectral wave model WAM6-GPU v1.0. **Geoscientific Model Development**, v. 17, n. 16, p. 6123–6136, 2024.

Rio de Janeiro, 1 de outubro de 2026.

Matheus Hoffmann Fernandes Santos - Aluno

A definir, Dr. - Orientador

Pedro Veras Guimarães, Dr. - Coorientador